



3.Static & Dynamic Testing

- 3.1. Static Techniques – Review
 - 3.1.1. Review Process (Informal & Formal)
 - 3.1.2. Technical or Peer Review
 - 3.1.3. Walkthrough
 - 3.1.4. Inspection
- 3.2. Static Techniques – Static Analysis
 - 3.2.1. Static Analysis by Tools (Automated Static Analysis)
- 3.3. Test Design Techniques: Black Box Testing
 - 3.3.1. Equivalence Partitioning
 - 3.3.2. Boundary Value Analysis
 - 3.3.3. Decision Table Testing
 - 3.3.4. State Transition Testing
- 3.4. Test Design Techniques -White Box Testing Techniques (coverage based and fault-based)
 - 3.4.1. Statement coverage
 - 3.4.2. Branch & Decision coverage
 - 3.4.3. Path coverage
 - 3.4.4. McCabe's Cyclomatic Complexity Metric (Computation of Cyclomatic Complexity to be covered)
 - 3.4.5. Data Flow based Testing
 - 3.4.6. Mutation Testing
- 3.5. Test Design Techniques -Experience based Techniques
 - 3.5.1. Error Guessing
 - 3.5.2. Exploratory Testing

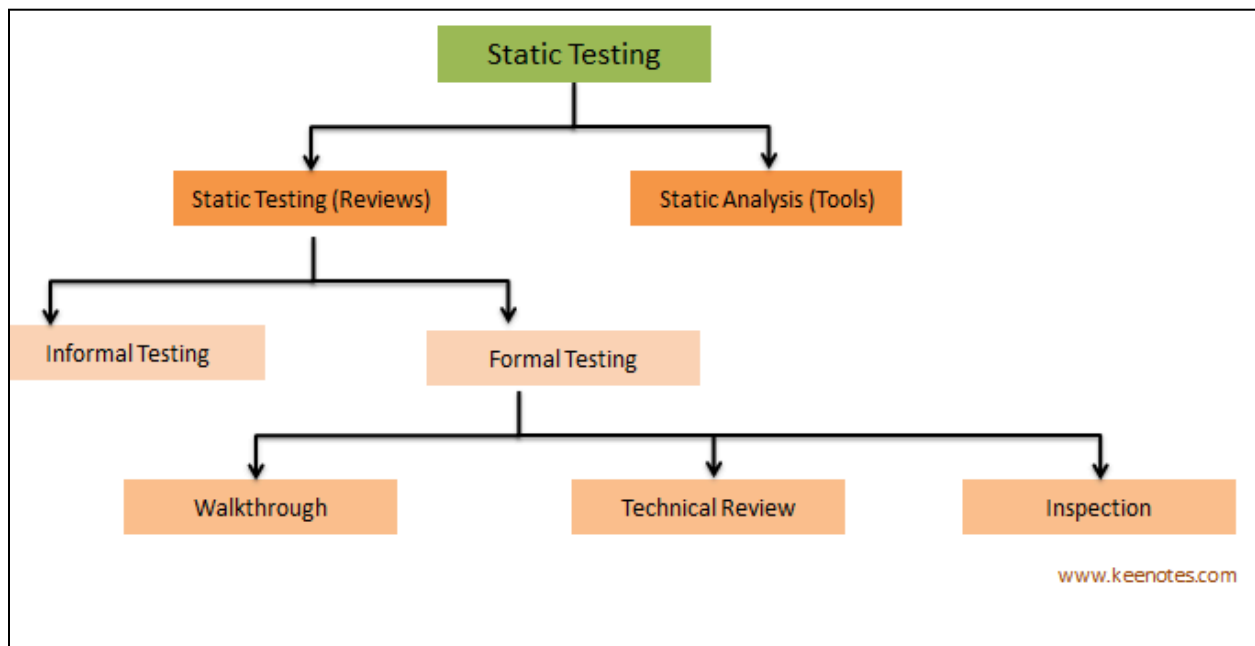
3.1. Static Techniques – Review

Static techniques focus on identifying defects without executing the code. These techniques help in early detection of errors, ensuring better software quality and reducing the cost of fixing bugs later in the development cycle. One of the most important static techniques is **review**.

Objectives of Static testing

The main objectives of performing static testing is as below:

- Static testing will identify, and fix the bugs at the earliest possible time.
- It is used to save both time and cost.
- It is used to identify defects in the early stage of SDLC, where we can fix them easily.



3.1.1. Review Process (Informal & Formal)

The **review process** involves examining software artifacts (like requirements, design, or code) to identify mistakes. Reviews can be either **informal** or **formal**:

- **Informal Review:**

- An unstructured review where team members casually go through the documents or code.
- Usually, there's no formal documentation or fixed process.

- Example: A developer sharing a piece of code with a teammate for quick feedback.

- **Formal Review:**

- A formal review is a structured process where team members carefully check documents or code to find mistakes. It includes specific roles like the author (who created the work), reviewers (who check for errors), and a scribe (who notes the issues found).
- The process usually involves planning the review, preparing by reading the materials, discussing the findings in a meeting, and following up to make sure corrections are made.
- **Example:** A team meeting to review the design of a new software feature, where everyone looks for problems, and the scribe records the issues in a report for future reference.

3.1.2. Technical or Peer Review

A **technical review**, also called a **peer review**, is a formal process where team members carefully check software-related documents or code to find mistakes and suggest improvements. This review is done by skilled team members, but **management doesn't take part** — the focus stays on technical quality, not on performance evaluation

Purpose: The main goals of a technical review are:

- To **find defects** in the code or documents early.
- To **ensure the work follows coding standards** and best practices.
- To **suggest improvements** for better performance, security, or readability.
- To **share knowledge** among team members, helping everyone understand the code better.

Participants: A typical technical review team includes:

- **Author:** The person who wrote the code or document being reviewed.
- **Reviewers:** One or more team members who check the work for errors and improvements.
- **Moderator:** Someone who leads the review, keeps the discussion focused, and ensures the process follows a clear structure.

Example: A team of developers reviewing the code for a new API to find any bugs, security issues, or areas where the code doesn't follow best practices.

3.1.3. Walkthrough

A **walkthrough** is a type of review where the author of a document or code explains their work to the team, step by step. The main goal is to help others understand the work, gather feedback, and find any mistakes.

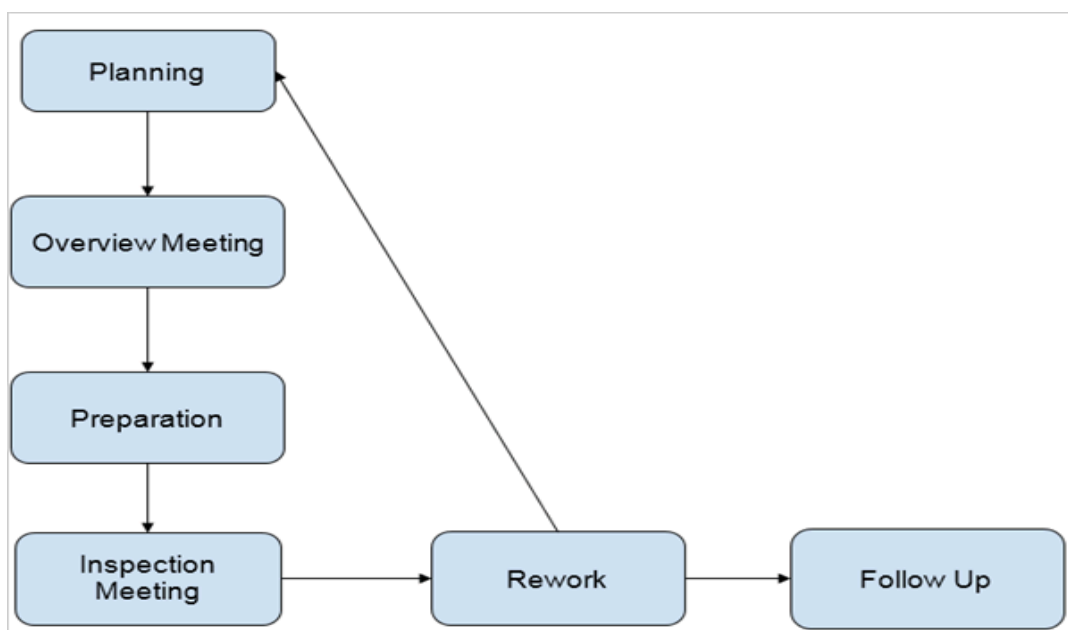
- **How it works:** The author presents their code, design, or document, explaining how it works, why certain decisions were made, and what each part does.
- **Participants:** Team members listen, ask questions, suggest improvements, and point out any errors they notice.
- A **walkthrough** is generally considered a **semi-formal** review process — it's more structured than an informal review but less rigid than a formal inspection.
- **Purpose:** To share knowledge, get fresh perspectives, and catch simple mistakes early.

Example: A developer shows their teammates the code for a new feature, explaining how it works and why they wrote it that way. The team asks questions, suggests small changes, and helps spot any errors — all in a relaxed, open discussion.

3.1.4. Inspection

Inspection is a formal process used to carefully check software documents or code to find mistakes. It's like a detailed group review where a team looks for errors in things like design plans, requirements, or code — **without running the program**.

Inspection Process:



1. Planning:

- The moderator organizes the inspection by defining the scope, goals, and schedule.
- Documents are shared with participants, and checklists may be provided to guide the review.

2. Overview: The author gives a brief explanation of the work being reviewed, helping participants understand the context.

3. Preparation:

- Each participant independently examines the document or code.
- They use checklists to identify defects like logic errors, security flaws, or non-compliance with standards.

4. Inspection Meeting:

- The team discusses the defects they found.
- The scribe records all identified issues but doesn't suggest solutions — the focus is purely on finding problems.

5. Rework: The author fixes the defects based on the feedback provided.

6. Follow-up: The moderator verifies whether the corrections were made and decides if a re-inspection is needed.

Who's involved / Roles in the Inspection Process:

- **Author:** The person who wrote the code or document.
- **Moderator:** Keeps the review on track and organized.
- **Reviewer/Inspector :** Team members who look for mistakes.
- **Scribe/ Recorder :** Writes down the issues found during the meeting.

3.2. Static Techniques – Static Analysis

Static Analysis is a method of checking a program's code without running it. It helps find mistakes, security problems, and areas where the code doesn't follow best practices. This process allows developers to fix issues early, making the software more secure and reliable before it's ever tested or used.

3.2.1. Static Analysis by Tools (Automated Static Analysis)

Automated static analysis means using special software tools to check a program's code without running it. These tools look for mistakes, security problems, and bad coding practices. The main goal is to catch and fix these issues early, while the program is still being built.

How Automated Static Analysis Works

The process generally involves the following steps:

1. **Code Parsing:** The tool reads the program's code line by line, breaking it down into smaller components for analysis.
2. **Rule Checking:** It checks the code against predefined rules or coding standards to identify violations.
3. **Data Flow Analysis:** The tool tracks how data moves through the program to detect logical errors, like null pointer dereferences or uninitialized variables.
4. **Reporting:** Finally, the tool generates a detailed report showing the errors, warnings, and suggestions for improvement.

Advantages of Automated Static Analysis

- **Early Detection of Bugs:** Finds issues at the coding stage, reducing debugging efforts later.
- **Enhanced Security:** Identifies potential security risks before deployment.
- **Consistency:** Ensures code follows organizational coding standards and best practices.
- **Time-Saving:** Automated tools work faster than manual code reviews.
- **Improved Maintainability:** Cleaner code is easier to understand, update, and expand.

Popular Static Analysis Tools

- **SonarQube:** An open-source tool for continuous inspection of code quality and security.
- **Checkmarx:** Focuses on identifying security vulnerabilities in code.
- **ESLint:** Used for identifying problematic patterns in JavaScript code.
- **Fortify:** Helps secure applications by analyzing source code.
- **Coverity:** Detects critical defects and security vulnerabilities.

Difference between Static Testing and Dynamic Testing

Aspect	Static Testing	Dynamic Testing
Meaning	Static Testing is a type of testing where the code is checked without executing the program.	Dynamic Testing is a type of testing where the program is executed to find errors.
Execution	In Static Testing, the program is not run.	In Dynamic Testing, the program is run.
Focus	Static Testing focuses on finding errors in code, design, and documents.	Dynamic Testing focuses on checking the behavior and output of the software.
When Performed	Static Testing is done in early stages of development.	Dynamic Testing is done after code is developed.
Techniques	Static Testing includes reviews, walkthroughs, and inspections.	Dynamic Testing includes unit testing, integration testing, system testing.
Errors Found	It finds syntax errors, design issues, and documentation errors.	It finds runtime errors, logical errors, and performance issues.
Who Performs	Static Testing is performed by developers and reviewers.	Dynamic Testing is performed by testers and developers.
Example	Example: Reviewing code or documents without running the program.	Example: Running a program and checking if output is correct.

3.3. Test Design Techniques: Black Box Testing

Black Box Testing is a software testing method where testers focus only on the software's functionality, without knowing how the internal code works. The main goal is to check whether the software behaves correctly by giving inputs and checking outputs.

There are several techniques used in Black Box Testing:

3.3.1. Equivalence Partitioning

Equivalence Partitioning is a method where input data is divided into groups (or partitions) that are

expected to produce the same result.

Instead of testing every single value, you pick just one value from each group, because all values in the same group should behave similarly.

Example:

Imagine a system where a user can enter their age, and valid ages are between **18 and 60**:

- **Valid partition:** Ages between **18 to 60** → Test with **30**
- **Invalid partitions:**
 - **Below 18** → Test with **10**
 - **Above 60** → Test with **70**

So, instead of testing every single age (like 18, 19, 20...), you test with: **10, 30, and 70** — covering all possible scenarios with fewer tests.

It reduces the number of test cases while still covering all possibilities.

3.3.2. Boundary Value Analysis

Boundary Value Analysis focuses on testing the boundaries (or edges) of input ranges, since errors often occur at these extreme points. Test the values at, just below, and just above the boundaries.

Example: For an input field that accepts numbers between **1 and 100**:

- **Lower boundary:** Test **0, 1, and 2**
- **Upper boundary:** Test **99, 100, and 101**

Test cases: **0, 1, 2, 99, 100, and 101**.

It helps catch off-by-one errors, like a system incorrectly accepting **101** when it should only allow **100**.

3.3.3. Decision Table Testing

Decision Table Testing is used when a system's output depends on multiple conditions or combinations of inputs.

It organizes inputs and expected outputs in a table format to ensure all combinations are tested.

Example:

A **login system** works like this:

- **Condition 1:** Correct username
- **Condition 2:** Correct password

Username	Password	Expected Result
Correct	Correct	Login Successful
Correct	Incorrect	Login Failed
Incorrect	Correct	Login Failed
Incorrect	Incorrect	Login Failed

It ensures you test **all possible combinations** of inputs, helping to catch any unexpected outcomes.

3.3.4. State Transition Testing

State Transition Testing checks how the system behaves when it moves from one **state** (or condition) to another based on inputs or events.

This technique is useful for systems with workflows, like ATMs or shopping carts.

Example:

Consider an **ATM machine**:

- **State 1:** Card inserted → **Enter PIN**
- **State 2:**
 - **Correct PIN** → **Access account**
 - **Incorrect PIN (3 times)** → **Card blocked**

Test cases:

1. Insert card → Enter correct PIN → Access account
2. Insert card → Enter incorrect PIN 3 times → Card blocked

It tests how the system reacts to **previous actions**, making sure the correct transitions happen.

3.4. Test Design Techniques -White Box Testing Techniques (coverage based and fault-based)

White Box Testing is a software testing method where testers check the internal structure, logic, and code of a program. It focuses on how the code works, rather than just what the software does.

It's also known as **Clear Box Testing**, **Glass Box Testing**, or **Structural Testing**.

There are two main approaches:

- **Coverage-based testing:** Ensuring certain parts of the code are executed during tests.
- **Fault-based testing:** Intentionally adding errors to test how the program responds.

3.4.1. Statement Coverage

- Statement Coverage checks if every line of code (statement) in the program is executed at least once during testing.
- The goal is to make sure no line of code goes untested.

Example:

```
def check_even(num):  
    if num % 2 == 0:  
        print("Even number")  
    else:  
        print("Odd number")
```

Test cases:

- **Input: 4** → covers the "Even number" statement
- **Input: 3** → covers the "Odd number" statement

Statement Coverage Formula:

$$\text{Statement Coverage} = \frac{\text{Number of statements executed}}{\text{Total number of statements}} \times 100$$

3.4.2. Branch & Decision Coverage

A) Branch Coverage (Also called Decision Coverage in some contexts)

Branch Coverage ensures that **each possible branch (True and False)** of every decision point is executed at least once.

In simple words: Every condition **must go both ways (true and false)** during testing.

Example:

```
def check_number(num):  
    if num > 0:  
        print("Positive")  
    else:  
        print("Negative or Zero")
```

Possible branches:

1. **True branch** → when $x > 0$
2. **False branch** → when $x \leq 0$

Test Cases:

- **Test Case 1:** $x = 5$ → True branch executed
- **Test Case 2:** $x = -2$ → False branch executed

Both branches are covered → 100% Branch Coverage

Goal: Ensure both **true** and **false** paths of the if condition are checked.

$$\text{Branch Coverage} = \left(\frac{\text{Number of executed branches}}{\text{Total number of branches}} \right) \times 100$$

B) Decision Coverage

Decision Coverage checks whether **each decision outcome (True/False)** has been executed.

It focuses on the **result of the condition**, not each individual condition.

Example:

```
if (x > 0 and y > 0):  
    print("Both Positive")  
else:  
    print("At least one is non-positive")
```

Decision:

- $(x > 0 \text{ and } y > 0)$ → can be:
 - True
 - False

Test Cases:

- Test Case 1: $x = 5, y = 10 \rightarrow \text{True}$
- Test Case 2: $x = 5, y = -3 \rightarrow \text{False}$

Both outcomes tested $\rightarrow$ **100% Decision Coverage**

3.4.3. Path Coverage

Path Coverage ensures that **all possible execution paths** in a program are tested at least once. It checks **every possible route** the program can take from start to end.

Example

```
if (x > 0):  
    print("A")  
    if (y > 0):  
        print("B")  
    else:  
        print("C")  
else:  
    print("D")
```

Possible Execution Paths

1. **Path 1:**
 $x > 0 \rightarrow \text{True}$
 $y > 0 \rightarrow \text{True}$
Output: A $\rightarrow$ B
2. **Path 2:**
 $x > 0 \rightarrow \text{True}$
 $y > 0 \rightarrow \text{False}$
Output: A $\rightarrow$ C
3. **Path 3:**
 $x > 0 \rightarrow \text{False}$
Output: D

Test Cases

Test Case	x	y	Path Covered
TC1	5	10	Path 1 (A → B)
TC2	5	-2	Path 2 (A → C)
TC3	-3	7	Path 3 (D)

All paths executed → **100% Path Coverage**

Formula

$$\text{Path Coverage} = \frac{\text{Number of paths executed}}{\text{Total number of paths}} \times 100$$

3.4.4. McCabe's Cyclomatic Complexity Metric

- McCabe's Cyclomatic Complexity measures how complex a program is by counting the number of independent paths in the code.
- More complexity means **higher chances of bugs** and **harder maintenance**.

Formula:

$$\text{Cyclomatic Complexity (V)} = E - N + 2P$$

Where:

- **E** = Number of edges (arrows) in the flowchart
- **N** = Number of nodes (decision points)
- **P** = Number of connected components (usually 1 for a single program)

3.4.5. Data Flow based Testing

Data Flow Testing is a **white-box testing technique** that focuses on how data moves through a program. It checks how variables are defined, used, and destroyed to catch errors like:

- **Uninitialized variables** (using a variable without giving it a value)

- **Unused variables** (defining a variable but never using it)
- **Incorrect updates** (changing a variable's value wrongly)

Example:

```
def calculate_sum(a, b):
    result = a + b          # 'result' is defined
    return result           # 'result' is used
```

Here:

- **Definition of variable:** result = a + b
- **Usage of variable:** return result

Data flow testing would check if the result is properly defined before use and not left uninitialized.

3.4.6. Mutation Testing

Mutation Testing is a method to check if your test cases are good enough. It works by **making small mistakes (mutations) in your code on purpose** — like changing a plus (+) to a minus (-) — and then running your tests again.

- If your tests **catch the mistake and fail**, that's a **good sign** — it means your tests are strong.
- If your tests **don't notice the mistake and still pass**, that's **bad** — it means your tests are weak and need to be improved.

Example:

Original code:

```
def multiply(a, b):
    return a * b
```

Mutation: Change * to +

```
def multiply(a, b):
    return a + b
```

If your test **fails** after a mutation — **your test is strong**.

If it **passes** despite the mutation — **your test is weak** and needs improvement.

Note: Students should refer to their handwritten notebook notes and Class Test 1 solutions for solved examples of White Box and Black Box Testing.

Difference between White Box Testing and Black Box Testing

Aspect	White Box Testing	Black Box Testing
Meaning	White Box Testing is a type of testing where the internal code and logic of the program are tested.	Black Box Testing is a type of testing where the functionality of the software is tested without knowing the internal code.
Knowledge of Code	In White Box Testing, the tester must have knowledge of the code.	In Black Box Testing, the tester does not need knowledge of the code.
Focus	White Box Testing focuses on internal structure like logic, conditions, and paths.	Black Box Testing focuses on input and output behavior of the system.
Performed By	White Box Testing is usually performed by developers or skilled testers.	Black Box Testing is performed by testers or end users.
Test Design	In White Box Testing, test cases are designed based on the code structure.	In Black Box Testing, test cases are designed based on requirements.
Objective	The objective of White Box Testing is to find logical and coding errors.	The objective of Black Box Testing is to check whether the system works as expected.
Level of Testing	White Box Testing is mainly used in unit testing.	Black Box Testing is mainly used in system and acceptance testing.
Example	For example, checking all if-else conditions in a program.	For example, checking login functionality with valid and invalid inputs.

3.5. Test Design Techniques - Experience-based Techniques

Experience-based testing techniques rely on the tester's knowledge, experience, and intuition to find defects. These techniques do not follow formal test cases or structured processes but focus on creatively exploring the software to uncover unexpected bugs.

3.5.1. Error Guessing

Error guessing is a testing technique where testers use their experience and understanding of the system to predict areas where defects are likely to occur. It is based on the belief that experienced testers can 'guess' potential errors based on past knowledge.

Key points:

- Relies on a tester's skill, intuition, and experience.
- No formal rules or procedures are followed.
- Focuses on common problem areas like input fields, boundary values, and error messages.

Examples:

1. **Invalid inputs:** Entering letters in a numeric field to check if the system handles it properly.
2. **Divide by zero:** Testing a calculator app by dividing a number by zero.
3. **Blank fields:** Leaving mandatory fields empty and checking if an error message appears.

3.5.2. Exploratory Testing

Exploratory testing is a hands-on approach where testers simultaneously design and execute tests without predefined plans. The goal is to explore the software's functionality, finding defects through creative and unscripted tests.

Key points:

- Test design and execution happen simultaneously.
- Encourages creativity and flexibility.
- Helps discover unexpected issues that scripted tests might miss.
- Can be guided by checklists or charters, but there are no fixed test cases.

Examples:

1. **Exploring a new app:** Clicking buttons, entering random data, and navigating through menus to see how the app responds.
2. **Testing a form:** Filling in valid and invalid data combinations without following a set sequence.
3. **Checking app responsiveness:** Rapidly switching between screens or resizing windows to check for layout issues.

Differences between Error Guessing and Exploratory Testing:

Aspect	Error Guessing	Exploratory Testing
Meaning	Tester guesses where errors can occur	Tester tests and learns the system at the same time
Approach	Based on guess and experience	Based on exploring the system
Planning	No planning needed	Little planning is done
Test Cases	No test cases	Test cases are created while testing
Documentation	Not written properly	Some notes can be written
Flexibility	Less flexible	More flexible
Focus	Find errors by guessing	Understand system and find errors
Skill Needed	Experience is important	Analytical thinking is important
Example	Trying wrong password to check error	Trying different inputs and checking system behavior